

Mock 2 — Classic Campus Round

C++ OOP · live coding · LogiFlow · timed

Mock interview

OOP + coding

Difference from Mock 1: Mock 1 assumes an architect who drills design. This one assumes a conventional panel that wants to confirm you know your language and can write correct code with your mouth moving. Both are plausible for a 20-minute slot at 10:20.

How to use. 25-minute timer. Read the panel line aloud, answer from memory. For the coding sections, **write the code by hand on paper** while narrating — do not type, and do not look at the printed solution until you have finished your own.

Intro to memorise (35–40 sec):

Good morning. I am Kavya Bhatiya, B.Tech in Artificial Intelligence at SVNIT Surat, CGPA 9.05, from Vadodara. I code in C, C++ and Python — 470-plus problems on LeetCode, max contest rating 1620. My strongest project is LogiFlow, a multimodal AI logistics platform on FastAPI, PostgreSQL and Next.js with a HistGradientBoosting delay-risk service, which placed in the Global Top 106 of the Google Solution Challenge 2026. I also work on multi-objective recommender research using NSGA-II. My coursework covers DSA, OOP, operating systems, computer networks and DBMS.

Rules. (1) Coding: restate the problem, state brute force with complexity, then optimise — never jump straight to the hash map silently. (2) OOP: give the one-line definition, then *one* concrete example, then stop. (3) Never say “I don’t remember” about a core subject — reason it out loud from first principles instead.

Clock

0:00–2:00	Intro	Self-introduction, subjects you are comfortable with.
2:00–8:00	C++ OOP	Copy constructor (deep vs shallow), virtual, polymorphism, possible dry-run.
8:00–14:00	LogiFlow deep dive	Your contribution, stack justification, model metrics, what broke.
14:00–20:00	Coding	Two Sum wrapped in a class, then an in-place array mutation.
20:00–23:00	DBMS / OS	Normalisation, index, ACID; process vs thread, deadlock.
23:00–25:00	Why GEHC + close	Motivation, location, your question.

Script

0:00 — Intro

INTERVIEWER Good morning. Please introduce yourself.

YOU SAY

YOU SAY

The 35–40 second intro above. End with the subject list — it invites them to pick, which means you control the menu.

INTERVIEWER Which subjects are you most comfortable in?

YOU SAY

YOU SAY

Data structures and algorithms first, then object-oriented programming in C++, then DBMS. I am comfortable with operating systems and networks at the concept level — I can reason about processes, scheduling and memory, and about the TCP and HTTP layers as I use them in my backends — but DSA and OOP are where I would rate myself strongest.

2:00 — C++ OOP fundamentals

INTERVIEWER What is a copy constructor, and when is it called?

YOU SAY

YOU SAY

A copy constructor creates a new object as a copy of an existing object of the same class. Its signature takes a reference to a const object of the class — `ClassName(const ClassName& other)`. It is called in four situations: when I initialise one object from another, when an object is passed to a function by value, when an object is returned by value, and when the compiler needs a temporary copy. The parameter must be a reference, because if it were taken by value, passing the argument would itself require a copy, which would call the copy constructor again — infinite recursion. The compiler will not accept it.

INTERVIEWER And if you do not write one, what does the compiler give you?

YOU SAY

YOU SAY

A default copy constructor that does a member-wise copy — a shallow copy. For a class holding only value members that is correct and I should not write my own. The problem is a class that owns a raw pointer: the default copies the pointer, not what it points to, so two objects now point at the same memory. Both destructors run `delete` on it, and that is a double free, plus any write through one object silently corrupts the other. That is when I need a deep copy — allocate my own buffer and copy the contents.

INTERVIEWER Show me. Write a class where the default copy would be wrong.

```
class Buffer {
    int* data;
    size_t n;
public:
    explicit Buffer(size_t n) : data(new int[n]), n(n) {}

    // Deep copy: allocate our own storage, then copy contents.
    Buffer(const Buffer& o) : data(new int[o.n]), n(o.n) {
        std::copy(o.data, o.data + o.n, data);
    }
};
```

```

    }

    // Copy assignment: guard self-assignment, free old, then copy.
    Buffer& operator=(const Buffer& o) {
        if (this == &o) return *this;
        int* tmp = new int[o.n];           // allocate BEFORE delete
        std::copy(o.data, o.data + o.n, tmp); // so a throw leaves us valid
        delete[] data;
        data = tmp;
        n = o.n;
        return *this;
    }

    ~Buffer() { delete[] data; }
};

```

YOU SAY

YOU SAY

Three things I would say while writing this, sir. One: if I need a custom destructor, I almost certainly need a custom copy constructor and copy assignment too — that is the Rule of Three. Two: in the assignment operator I allocate the new buffer before deleting the old one, so if allocation throws, the object is still in a valid state. Three: in modern C++ the better answer is to not own raw memory at all — use `std::vector` or a smart pointer and the compiler-generated copy becomes correct, which is the Rule of Zero. I wrote it manually because you asked me to show where the default breaks.

INTERVIEWER *What is polymorphism? Types?*

YOU SAY

YOU SAY

One interface, many behaviours. Two kinds. Compile-time polymorphism, which is function overloading, operator overloading and templates — the compiler resolves which version to call from the argument types, so it costs nothing at runtime. And runtime polymorphism, which is achieved through virtual functions and inheritance — I hold a base class pointer or reference, and the call dispatches to the derived override based on the object's actual type at run time, through the vtable.

INTERVIEWER *Explain virtual with an example, and tell me what happens without it.*

```

struct Shape {
    virtual double area() const = 0;    // pure virtual -> Shape is abstract
    virtual ~Shape() = default;        // MUST be virtual: base ptr delete
};

struct Circle : Shape {
    double r;
    explicit Circle(double r) : r(r) {}
    double area() const override { return 3.14159 * r * r; }
};

```

```

struct Rect : Shape {
    double w, h;
    Rect(double w, double h) : w(w), h(h) {}
    double area() const override { return w * h; }
};

// The point: this loop never mentions Circle or Rect.
double total(const std::vector<Shape*>& v) {
    double s = 0;
    for (const Shape* p : v) s += p->area();    // virtual dispatch
    return s;
}

```

YOU SAY

YOU SAY

Without `virtual` on `area`, the call through `Shape*` would bind statically to `Shape::area` — every shape would report the base version, and the whole hierarchy would be pointless. And the virtual destructor is not optional: if I delete a `Circle` through a `Shape*` and the destructor is non-virtual, only the base destructor runs, so any resource the derived class owns leaks. The practical value of this is in the `total` function — it does not name a single concrete shape, so adding a triangle tomorrow does not touch it. I used exactly this idea in LogiFlow: transport modes sit behind a common interface so the ranking code does not branch on mode.

INTERVIEWER *One dry-run. If I call a virtual function from inside a base class constructor, which version runs?*

YOU SAY

YOU SAY

The base class version, not the override. During base construction the derived part of the object does not exist yet, so the vtable pointer still refers to the base class — dispatch cannot reach an override that would touch uninitialised derived members. That is why calling virtual functions from constructors and destructors is a known trap; the usual fix is two-phase initialisation, construct then call an `init` that can safely dispatch.

8:00 — LogiFlow deep dive

INTERVIEWER *Tell me about your best project in detail.*

YOU SAY

YOU SAY

LogiFlow. Freight between two cities usually has more than one viable way — road, rail, air, sea, or a combination — and the choice is a trade-off among delivery time, reliability, cost and sustainability. LogiFlow takes the request and constraints and returns ranked options so the planner sees the trade-offs. Backend is FastAPI with PostgreSQL; front-end is Next.js; Gemini assists on the product surface; delay risk comes from a HistGradientBoostingClassifier — 73 percent accuracy, 68 percent delayed-shipment recall, 78.3 percent ROC-AUC. Docker is how we package the service. It reached the Global Top 106 in the Google Solution Challenge 2026.

INTERVIEWER *What is your specific contribution, and what did you find hardest?*

YOU SAY

YOU SAY

I worked on the backend and the model side — the FastAPI service, the Postgres schema for corridors and shipment history, the scoring and ranking logic, and training and integrating the delay classifier. The hardest part was not the model, it was making the model's output mean something in the product. My first version returned a raw probability, and a probability on its own is not a decision — so I had to normalise cost, time and delay risk onto a comparable scale and expose weights so a deadline-driven user and a cost-driven user get different orderings from the same data. Getting that normalisation to not be arbitrary took the most iterations.

INTERVIEWER *You said multimodal. What does that mean here?*

YOU SAY

YOU SAY

Two things, and I should separate them. In the logistics sense, multimodal means a shipment can combine transport modes — rail for the long haul, road for first and last mile. In the input sense, the system accepts more than text: a user can upload a document or image of shipment details and we use Gemini to extract structured fields instead of making them type it in. That extraction sits on the ingestion path only — if it fails, manual entry still works and the ranking is unaffected.

INTERVIEWER *And the delay model — what accuracy did you get?*

YOU SAY

You say. Seventy-three percent accuracy, but I would not want to give you only that number. The classes are imbalanced — most shipments are on time — so accuracy flatters any model that just predicts “on time”. The number that matters is recall on the delayed class, which is sixty-eight percent, meaning I catch about two-thirds of genuine delays, and ROC-AUC of 0.783, which shows real separation against 0.5 for random. The model is histogram-based gradient boosting; I benchmarked it against a logistic regression baseline so I could tell whether the extra complexity was earning anything. It is shipped as advisory risk feeding a ranking, never as a promise that a shipment will be late.

INTERVIEWER Any bug you remember debugging?

YOU SAY

YOU SAY

Yes — a container worked locally and failed after deploy. The model artefact path existed on my machine but was not copied into the image, so the process started and prediction requests failed. Fix: copy the artefact in the Docker build, load once at startup, and fail loud at boot. Lesson: “works on my machine” is about my filesystem, not my code.

14:00 — Coding

INTERVIEWER Simple one. Given an array of integers and a target, return the indices of the two numbers that add up to the target. Since we have been talking OOP, wrap it in a class.

YOU SAY

YOU SAY

Restating: one array, one target, return the two indices, and I will assume exactly one valid pair unless you tell me otherwise. Brute force is two nested loops — $O(n^2)$ time, $O(1)$ space. I can do better by trading space for time: as I walk the array once, for each element I compute what value I still need, and check whether I have already seen it. A hash map from value to index gives me that lookup in $O(1)$ average, so the whole thing is $O(n)$ time and $O(n)$ space. Let me write it.

```
class TwoSum {
public:
    // O(n) time, O(n) space. Single pass.
    std::vector<int> solve(const std::vector<int>& nums, int target) const {
        std::unordered_map<int,int> seen; // value -> index
        for (int i = 0; i < (int)nums.size(); ++i) {
            int need = target - nums[i];
            auto it = seen.find(need);
            if (it != seen.end())
                return { it->second, i }; // earlier index first
            seen[nums[i]] = i; // insert AFTER lookup
        }
        return {}; // no valid pair
    }
};
```

YOU SAY

YOU SAY

Three points I would narrate as I write. First, I insert into the map *after* the lookup, not before — if I inserted first, an element could pair with itself when `target` is exactly twice that element. Second, the map stores value to index, not index to value, because I search by value. Third, duplicates are safe: if the same value appears twice, the second occurrence finds the first in the map before it overwrites it.

Complexity is $O(n)$ average; worst case $O(n^2)$ in a pathological hash-collision scenario, which is theoretical here.

INTERVIEWER Dry-run it for me on `[2, 7, 11, 15]`, target `9`.

Say this out loud, index by index:

`i=0`, `nums[0]=2`, `need = 9-2 = 7`. Map is empty, so no hit. Insert `{2 -> 0}`.

`i=1`, `nums[1]=7`, `need = 9-7 = 2`. Map has `2` at index `0`. Hit — return `{0, 1}`.

Second case to volunteer: `[3, 3]`, target `6`. `i=0`: need `3`, map empty, insert `{3 -> 0}`. `i=1`: need `3`, found at `0`, return `{0, 1}`. This is the case that breaks a naive insert-first version.

Third: `[1, 2]`, target `100` — loop finishes, return empty vector.

INTERVIEWER Now a follow-up. Same array, but suppose it is already sorted and I do not want extra space.

YOU SAY

YOU SAY

Then two pointers, sir — one at each end. If the sum is too large I move the right pointer left, because everything to its right is larger and cannot help; if the sum is too small I move the left pointer right. $O(n)$ time, $O(1)$ space. It only works because sorted order tells me which direction reduces the sum, which is exactly the information the unsorted version lacks — and that is why the unsorted version has to pay memory instead.

```
std::vector<int> twoSumSorted(const std::vector<int>& a, int target) {
    int l = 0, r = (int)a.size() - 1;
    while (l < r) {
        long long s = (long long)a[l] + a[r];    // widen: avoid overflow
        if (s == target) return { l, r };
        if (s < target) ++l; else --r;
    }
    return {};
}
```

INTERVIEWER One more, in-place. Move all zeroes in an array to the end, keeping the order of the non-zero elements. No extra array.

YOU SAY

YOU SAY

Two-pointer write-index pattern. I keep a write position starting at zero, scan with a read pointer, and every time I see a non-zero element I swap it into the write position and advance the write pointer. When the scan finishes, everything before the write pointer is the non-zero elements in their original order, and everything after is zeroes. One pass, $O(n)$ time, $O(1)$ extra space, and the swap version writes each element at most once.

```
void moveZeroes(std::vector<int>& nums) {
    int w = 0; // next write position
    for (int r = 0; r < (int)nums.size(); ++r) {
        if (nums[r] != 0)
            std::swap(nums[w++], nums[r]); // w never overtakes r
    }
}
// [0,1,0,3,12] -> r=0 zero skip | r=1 swap(w0,r1) -> [1,0,0,3,12], w=1
//               -> r=2 zero skip | r=3 swap(w1,r3) -> [1,3,0,0,12], w=2
//               -> r=4 swap(w2,r4) -> [1,3,12,0,0], w=3
```

20:00 — DBMS and OS

INTERVIEWER *You used Postgres. What is normalisation and why do it?*

YOU SAY

YOU SAY

Organising a schema so each fact is stored once, by removing redundancy through decomposition into related tables with keys. First normal form means atomic column values, second means no partial dependency on part of a composite key, third means no transitive dependency — non-key attributes depend on the key and nothing else. The reason is update anomalies: if a carrier's name is duplicated across ten thousand shipment rows, one rename means ten thousand updates and any missed row is now a contradiction. In LogiFlow, corridors, carriers and shipments are separate tables joined by foreign keys for exactly that reason. The honest counterpoint is that normalisation costs joins, so for a read-heavy analytics query I would deliberately denormalise or keep a derived table — normalise by default, denormalise with a measured reason.

INTERVIEWER *What is an index, and why not index every column?*

YOU SAY

YOU SAY

An index is a separate structure, typically a B-tree, that lets the engine find rows matching a predicate without scanning the whole table — turning a linear scan into a logarithmic lookup. You do not index everything because every index must be updated on insert, update and delete, so writes get slower and storage grows. Indexes also do not help low-cardinality columns much — an index on a boolean rarely beats a scan. I index what appears in `WHERE` and `JOIN` conditions on large tables and nothing else.

INTERVIEWER *ACID — one line each.*

YOU SAY

YOU SAY

Atomicity: a transaction is all or nothing, no partial effect survives. Consistency: it moves the database from one valid state to another, respecting constraints. Isolation: concurrent transactions do not observe each other's intermediate state — how strictly is the isolation level. Durability: once committed, it survives a crash, which is what the write-ahead log guarantees. The one I would emphasise for a logistics system is atomicity — a tariff update that half-applies is worse than one that fails outright.

INTERVIEWER *Process versus thread.*

YOU SAY

YOU SAY

A process has its own address space; threads live inside a process and share its address space, code and data while each keeping its own stack, registers and program counter. Consequences: threads communicate cheaply through shared memory but can corrupt each other, so they need synchronisation, and one thread crashing can take the process down. Processes are isolated and safer, and communication costs an IPC mechanism. Context switching between threads is cheaper because the memory map does not change.

INTERVIEWER *Deadlock — conditions, and how do you handle it?*

YOU SAY

YOU SAY

Four conditions must hold simultaneously: mutual exclusion, hold and wait, no preemption, and circular wait. Break any one and deadlock cannot occur. In practice the approaches are prevention — for instance imposing a global ordering on lock acquisition so a cycle is impossible, which is the one I would actually use; avoidance, as in Banker's algorithm, when resource needs are known in advance; or detection and recovery, where you build a wait-for graph, look for a cycle, and kill or roll back a victim. Databases usually detect and abort one transaction, which is why application code must be prepared to retry.

23:00 — Why GE HealthCare, and close

INTERVIEWER *Why GE HealthCare? And why the Edison internship specifically?*

YOU SAY

YOU SAY

Two honest reasons. The first is the nature of the software. Almost everything I have built has been a model sitting inside a real system, and the part I find genuinely interesting is the system — the API, the schema, the deployment, what happens when the model is wrong. Edison is exactly that at a scale I cannot reach on my own: AI that has to run inside clinical workflows, where being wrong has a cost, so there is no room for a model that only looks good on a slide. That constraint is the reason I want to work there rather than a place where the model is the demo. The second is that I want to be forced to raise my standards on validation and reliability. On LogiFlow I have already run into the difference between a metric and a decision, and I would like to learn how that is done properly, by people who have done it for twenty years.

INTERVIEWER *You have research and hackathon wins. Would you not rather do a PhD or a startup?*

YOU SAY

YOU SAY

Research taught me how to be rigorous about comparisons, and I want to keep that habit, but what I actually enjoy is shipping — and I would rather learn engineering discipline inside a team that has real users and real constraints than optimise a benchmark on my own. If I pursue research later it will be better research for having built production systems first. Right now, an internship where I own a real component is the highest-value thing I can do.

INTERVIEWER *You are from Vadodara, studying in Surat. Any issue with Bengaluru?*

YOU SAY

YOU SAY

None at all. I am happy to relocate — no constraint on my side.

INTERVIEWER *Any questions for us?*

YOU SAY

YOU SAY

Yes — what does the first month look like for an Edison intern? Specifically, whether I would be reading an existing codebase or starting something new, because if it is the former I would like to prepare for that deliberately. And is there a language or framework the team uses that I should get ahead on before joining?

INTERVIEWER *Thank you Kavya.*

YOU SAY

YOU SAY

Thank you sir, I appreciated the questions.

Score this mock

Rate each area /10. Pass bar: 7+ average, and nothing below 5.

Intro	Under 45 s? LeetCode + LogiFlow + one number? Ended with subject menu?	–
C++ OOP	Copy ctor: const-ref reason + shallow/deep + Rule of Three, unprompted?	–
Virtual	Static vs dynamic binding explained? Virtual destructor mentioned yourself?	–
Project	Own contribution stated honestly? Volunteered 0.68 recall before being asked?	–
Coding	Brute force + complexity stated <i>before</i> optimising? Code compiled in your head?	–
Dry run	Walked [2, 7, 11, 15] index by index? Volunteered [3, 3] edge case?	–
DBMS / OS	Normalisation with the anomaly reason? Deadlock: all four conditions?	–
Why GEHC	Specific to Edison and clinical constraint, not generic “great company”?	–

Pass bar for tomorrow. You wrote Two Sum without a syntax hesitation and caught the insert-after-lookup point yourself. You said “Rule of Three” and “virtual destructor” before being prompted. You gave the recall number without being pushed. Your “why GEHC” mentioned a constraint, not a compliment.

Five things that lose this round. (1) Jumping to the hash map with no brute-force baseline. (2) Reciting “abstraction, encapsulation, inheritance, polymorphism” as a list with no example. (3) Saying accuracy 73 percent and stopping. (4) “I forgot” on deadlock or ACID — reason it out instead. (5) A generic “why GEHC” about brand and culture.

Last 10 minutes before the slot. Re-read only three things: your 40-second intro, the 73 / 0.68 / 0.783 explanation, and the copy constructor answer. Nothing else.